

Beyond E2E Scripts: Using LLM-Proposed Scenarios Without Letting the LLM Be the Oracle

Draft manuscript

May 2026

Abstract

Large language models can help testers explore application states that are not covered by ordinary end-to-end scripts, but they are unsafe as test oracles. They can suggest plausible scenarios, misread application semantics, and overstate weak evidence. Bayesilisk separates these roles. App-specific connectors publish observable context, available actions, and deterministic rules. First, Bayesilisk expands the supplied scenario space using connector-declared rules, action graphs, Grassmann attention, and optional model proposals. Candidate scenarios are then validated against connector-declared context and rules; malformed or unsupported proposals are rejected. Valid executable probes are prioritized with a Bayesian scoring procedure and run by the connector. Finally, Bayesilisk decides pass/fail with a deterministic ruleset over the returned evidence.

1 Introduction

End-to-end suites are precise, reproducible, and easy to trust when they fail, but their coverage is bounded by the scenarios a test author had time to enumerate. LLM-driven agents have the opposite shape: they can suggest unusual states and longer workflows, but they are unreliable when asked to decide whether the observed behavior is correct. The central design claim behind Bayesilisk is that these jobs should be separated. Models and coding agents may help expand the scenario space; deterministic verifier rules must remain the oracle.

This separation also changes what a test connector is expected to do. A connector should not embed Bayesilisk-specific app logic, and Bayesilisk should not know Cal.com-specific semantics. Instead, the connector publishes the observable surface of the app: routes, actions, state facts, identifiers, fixtures, expected behavior, and the evidence observed after a probe executes. Bayesilisk then generates candidate scenarios, rejects candidates that do not fit the connector contract, prioritizes executable probes, and verifies returned evidence against deterministic invariants. Each concrete scenario can be sampled from the declared scenario space with priority informed by proximity to contract boundaries, selected invariant planes, and suspected violations.

The intended workflow is:

1. A connector publishes app-specific context: routes, actors, identifiers, available actions, state facts, nearby tests, and expected behavior. This context defines the executable surface that Bayesilisk is allowed to reason about.
2. Bayesilisk constructs a candidate scenario space from connector-declared rules, mutation operators, action graphs, and optional model-proposed scenarios. LLM output is treated only as candidate input, never as an oracle.
3. Candidate scenarios are validated against the connector contract. A scenario that references unknown routes, actions, identifiers, invariants, missing state facts, unsupported parameter bindings, or undeclared expected behavior is rejected as malformed or unsupported.
4. Valid executable probes are prioritized. Grassmann-style attention routes search toward nearby invariant planes and contract boundaries, while Bayesian scoring ranks candidates by estimated fault probability.

5. The connector executes selected probes through the real app, a local test harness, or Playwright/API fixtures. The connector reports observed evidence such as statuses, URLs, state transitions, traces, screenshots, and execution errors. It does not decide whether the behavior is a bug.
6. Bayesilisk verifies the returned evidence against deterministic invariants. Only evidence-bearing probes that violate declared rules become issue-ready findings; malformed proposals and unsupported executions are recorded as proposer or connector telemetry, not product bugs.

The connector is app-specific. The Bayesilisk core is not. This boundary is the main architectural point: app teams write connectors that expose context, actions, and evidence; Bayesilisk provides the generic scenario expansion, validation, prioritization, and deterministic verification loop.

2 Architecture

The architecture has four trust boundaries.

1. **Scenario proposal is untrusted.** Connector-declared mutation rules, connector action graphs, and optional LLM proposals all produce candidate scenarios. A candidate is not yet an executable probe, and it is never an oracle.
2. **Validation is local and deterministic.** Before execution, Bayesilisk checks that every candidate references declared routes, actions, parameters, state facts, and invariants. Unsupported candidates are rejected and recorded as proposer telemetry.
3. **Execution belongs to the connector.** The connector maps a valid proposal to concrete app operations: Playwright navigation, API calls, fixture setup, database state, login/session actions, screenshots, or traces. The connector returns evidence; it does not decide pass/fail.
4. **Verification belongs to Bayesilisk.** Bayesilisk compares returned evidence with deterministic invariants. Only evidence-bearing violations become issue-ready findings.

Grassmann-style attention and Bayesian scoring operate between validation and execution. They choose which valid probes to run first; they do not decide truth. This matters for LLM integration. A weak model may produce an interesting-looking scenario that references an unavailable action or an undeclared invariant. Bayesilisk should reject such a candidate as malformed or unsupported, not send it to the app and not report it as a product bug.

3 Cal.com Case Study

We evaluated the current connector architecture on a local Cal.com checkout. The tested repository and revision were:

- Repository: <https://github.com/calcom/cal.com>
- Commit: 180ede28f0bddf2738933a6e60a8e80f6116d7da
- Commit date: 2026-05-14 19:30:21 +0000
- Commit subject: font-stack fallback fix for non-Latin script support (#29346)
- Bayesilisk seed: 150

The connector supplied route facts for Cal.com booking pages and an action graph for a longer booking workflow. From this context, Bayesilisk generated seven probe proposals. Six probes mutated unknown or stale `rescheduleUid` values on public, private, and dynamic booking routes. The seventh probe was a generated workflow sequence: create a booking, cancel it, then replay the cancelled booking UID through the public booking route as `reschedule` context.

Artifact	Path
Connector code	<code>examples/calcom/bayesilisk-probes.e2e.ts</code>
Source context	<code>examples/calcom/source-context.json</code>
Generated proposals	<code>examples/calcom/generated-proposals.json</code>
Sequence context	<code>examples/calcom/sequence-source-context.json</code>
Generated sequence	<code>examples/calcom/generated-sequence-proposals.json</code>
Execution evidence	<code>examples/calcom/execution-context.json</code>
Bayesilisk report	<code>examples/calcom/reports/report.json</code>
Issue payloads	<code>examples/calcom/reports/issue-payloads.json</code>
Upstream outcomes	<code>examples/calcom/upstream-outcomes.md</code>

Table 1: Checked-in Cal.com artifacts.

Area	Mutation	Expected	Observed
Public booking page	unknown id	404	200
Public booking page	stale id	404	200
Private booking link	unknown id	404	200
Private booking link	stale id	404	200
Dynamic booking page	unknown id	404	200
Dynamic booking page	stale id	404	200
Cancelled booking replay	cancelled uid	409	200

Table 2: Verified Cal.com connector observations.

The checked-in Bayesilisk artifacts are in `examples/calcom/`. They include the connector, source contexts, generated proposals, consolidated execution context, app-only JSON report, app-only Markdown report, issue payloads, and the two Playwright run logs for the clean rerun.

All seven proposals executed against local Playwright fixtures. All seven returned evidence that violated the connector-declared invariant: the route mutations expected semantic status 404 but observed 200; the cancelled-booking replay expected semantic status 409 but observed 200. The pass/fail verdict did not come from a model and did not come from the connector. The connector returned concrete evidence; Bayesilisk compared `expectedStatus` and `observedStatus`, classified the findings as `breakage.context-observed`, and emitted issue-ready payloads.

4 Baseline Comparison

The Cal.com evaluation also included comparison runs against nearby E2E tests, manual Playwright-style probing, weak local model proposal generation, and a browser-driving LLM-agent baseline. These comparisons are not used as the oracle; they are included to explain why the Bayesilisk split is useful.

The nearby upstream E2E tests did not expose the seven Bayesilisk findings. The Playwright-only connector checks did expose part of the same bug family, but they did not produce the full unknown/stale Cartesian expansion across all declared routes. The weak-model proposal run is a negative result: plausible model text did not satisfy the verifier gates, so it was rejected before it could become an app finding. The browser-driving LLM-agent baseline illustrates the oracle problem directly: in repeated local trials, the agent sometimes selected malformed or unusable actions and sometimes disagreed with the deterministic expected-vs-observed verdict. This is the behavior Bayesilisk is designed to contain.

5 Upstream Response

Two upstream issues were opened from this evidence: one for public `rescheduleUid` replay [1] and one for cancelled booking UID replay [2]. The first issue already has a human-authored fix pull request [3]. This

Workflow	Observation
Nearby E2E	14 nearby specs ran without unexpected failures; the existing suite did not cover these states.
Playwright-only probing	11 observations produced 4 failures; manual probes exposed only a subset of the bug family.
Bayesilisk route proposals	6 generated route proposals produced 6 findings across declared route variants.
Bayesilisk sequence proposal	1 generated workflow sequence produced 1 cancelled-booking replay finding.
Weak local model proposals	3 raw proposals produced 0 accepted probes because verifier validation rejected malformed candidates.
LLM-agent browser run	10 trials produced 3 oracle mismatches before deterministic verification.

Table 3: Cal.com evaluation summary.

is stronger validation than issue closure alone: an issue can be closed as erroneous, but a targeted fix PR indicates that a human contributor found the report actionable.

The upstream response is important because the claim is not only that Bayesilisk can emit plausible reports. The reports need to be useful to humans working on the target system.

Finding group	Issue	Human response
Public unknown/stale <code>rescheduleUid</code>	#29399	Open with fix PR #29400 at capture time
Private-link <code>rescheduleUid</code>	#29399 comment	Folded into the existing thread to avoid duplicate reporting
Cancelled booking UID replay	#29407	Open as a separate state-transition report at capture time

Table 4: Upstream response to Bayesilisk-generated Cal.com findings.

The private-link finding is tracked as a related comment rather than a separate issue to avoid duplicating the existing `rescheduleUid` thread. The cancelled-booking workflow finding is separate because it comes from a different state transition: a real booking is created, cancelled, and then replayed as public reschedule context.

6 Discussion

The evidence supports a concrete but bounded claim. Given connector-supplied source facts, rules, and action graphs, Bayesilisk can expand a scenario space, reject malformed proposals, prioritize executable probes, run them through an app-specific connector, and verify evidence without letting an LLM decide the outcome. This is different from a plain Playwright script because the connector does not enumerate every final generated probe by hand; it exposes the local state and rule space, and Bayesilisk produces the concrete mutations and the bounded workflow sequence.

The current sequence proposer is intentionally limited. It composes short sequences from declared actions and state facts; it does not synthesize arbitrary free-form browser sessions. That limitation is a design boundary, not a defect: the connector remains a small executor, while the core remains app-agnostic. A stronger generality claim will require additional real-app artifacts with the same discipline: exact repository revision, connector code, context JSON, generated proposals, execution evidence, deterministic report, issue payloads, and upstream response where available.

7 Conclusion

Bayesilisk treats LLMs as scenario proposers, not as test oracles. The Cal.com case study shows the architecture working on a real app checkout: seven generated probes, seven connector-observed failures, deterministic issue payloads, and an upstream human-authored fix PR for one reported finding. The broader lesson is that LLMs can help search the scenario space, but the oracle must remain a deterministic verifier over evidence returned by an app-specific connector.

References

- [1] Bayesilisk contributors. Cal.com issue 29399: Booking page opens instead of 404 for missing rescheduleuid. <https://github.com/calcom/cal.diy/issues/29399>, 2026. Issue report generated from Bayesilisk evidence.
- [2] Bayesilisk contributors. Cal.com issue 29407: Cancelled booking uid replay opens public booking route. <https://github.com/calcom/cal.diy/issues/29407>, 2026. Issue report generated from Bayesilisk workflow-sequence evidence.
- [3] Cal.com contributor. Cal.com pull request 29400: Return 404 for unknown rescheduleuid instead of opening booking flow. <https://github.com/calcom/cal.diy/pull/29400>, 2026. Human-authored fix pull request responding to Bayesilisk evidence.